# UART 16550

Baud rate = number of symbols sent per second

In most UART systems:

1 symbol = 1 bit

So practically,

Baud rate ≈ bits per second (bps)

Example

If baud rate = 9600

- 9600 bits are sent every second

If baud rate = 2 Mbps

- 2,000,000 bits are sent every second

Baud Rate = $\frac{fclk}{16*(PSD+1)*DL}$

**Example-1: External Baud Clock = 32 MHz**

$$2{,}000{,}000 = \frac{32{,}000{,}000}{16 \times (PSD + 1) \times DL}$$

$$16 \times (PSD + 1) \times DL = 16$$

Valid values:

- $PSD = 0 \Rightarrow (PSD + 1) = 1$
- $DL = 1$

$$\Rightarrow BR = \frac{32{,}000{,}000}{16 \times 1 \times 1} = 2{,}000{,}000$$

✅ **Exact 2 Mbps**

```
#define UART_BASE 0x43C00000
#define UART_LCR   (*(volatile unsigned int *)(UART_BASE + 0x0C))
#define UART_DLL   (*(volatile unsigned int *)(UART_BASE + 0x00))
#define UART_DLM   (*(volatile unsigned int *)(UART_BASE + 0x04))
#define UART_PSD   (*(volatile unsigned int *)(UART_BASE + 0x20))

int main()
{
    // Enable DLAB
    UART_LCR = 0x80;

    // Divisor for 2 Mbps
    UART_DLL = 1;
    UART_DLM = 0;

    // Prescaler
    UART_PSD = 0;

    // 8-bit, no parity, 1 stop
    UART_LCR = 0x03;

    while(1);
}
```

```
#define UART_LCR   (*(volatile unsigned int *)(UART_BASE + 0x0C))

#define UART_DLL   (*(volatile unsigned int *)(UART_BASE + 0x00))

#define UART_DLM   (*(volatile unsigned int *)(UART_BASE + 0x04))

#define UART_PSD   (*(volatile unsigned int *)(UART_BASE + 0x20))
```

These are **memory-mapped UART registers**.

| Register | Offset | Purpose |
|---|---|---|
| DLL | 0x00 | Lower 8 bits of baud-rate divisor |
| DLM | 0x04 | Upper 8 bits of baud-rate divisor |
| LCR | 0x0C | Line Control → data bits, parity, stop bit, DLAB |
| PSD | 0x20 | Prescaler → divides input clock |

**volatile** → tells the compiler:

"This is a hardware register, do NOT optimize or cache it."

Here is the **clear and correct English version** of that explanation:

**This is not a normal variable.**
**It is a hardware register.**
**Its value can change at any time.**
**Therefore, do not cache or optimize it.**
**Always read from and write to the actual hardware.**

---

🔹 3️⃣ **MAIN FUNCTION START**

```
int main()

{
```

• Program execution **starts from here**

---

🔹 4️⃣ **DLAB ENABLE**

```
UART_LCR = 0x80;
```

• 0x80 = 1000 0000 (binary)
• **Bit-7 = DLAB (Divisor Latch Access Bit)**
• DLAB = 1 → CPU can write **DLL/DLM** (baud-rate registers)
• DLAB = 0 → normal TX/RX operation

💡 **Analogy**
• DLL/DLM = speed knob
• DLAB = door to the knob
→ 1 = door open, 0 = door closed

🔹 5️⃣ **SET DIVISOR FOR 2 Mbps**

```
UART_DLL = 1;

UART_DLM = 0;
```

**Baud-rate formula:**

Baud = Fclk / (16 × (PSD + 1) × DL)

- Fclk = 32 MHz
- PSD = 0
- DL = 1

👉 Baud = **2 Mbps**

- DLL = lower 8 bits = 1
- DLM = upper 8 bits = 0

✅ UART baud rate set to **2 Mbps**

---

🔹 6️⃣ **PRESCALER**

UART_PSD = 0;

- PSD = 0 → divide factor = **1**
- Clock is **not slowed down**
- Exact baud rate achieved

---

🔹 7️⃣ **LINE FORMAT SET**

UART_LCR = 0x03;

- 0x03 = 0000 0011 (binary)

| Bit | Meaning |
|---|---|
| Bit-7 | DLAB = 0 (gate closed) |
| Bits 0–1 | 11 → 8-bit data |
| Parity | Disabled |
| Stop bit | 1 stop bit |

✅ UART format = **8N1**
(8 data bits, No parity, 1 stop bit)

---

🔹 8️⃣ **INFINITE LOOP**

while(1);

- Keeps program running forever
- UART configuration remains active
- Very common in embedded systems

---

🔹 🔥 **FINAL SUMMARY**

1. UART base address is defined
2. DLAB = 1 → enable baud-rate configuration
3. DLL/DLM set → baud rate = **2 Mbps**
4. Prescaler = 0 → no extra clock division
5. Line format set to **8N1**
6. Infinite loop keeps UART ready for use

# ZYNQ 7000

$$\text{Baud Rate} = \frac{ref\ clk}{CD*(BDIV+1)}$$

**1. CD (Clock Divisor)**

- **What it is:** It is the "Clock Divisor" value stored in the **BAUDGEN** (Baud Rate Generator) register.
- **Function:** This is the **first stage** of division. It takes the incoming Reference Clock (usually 100MHz) and divides it by the value of CD.
- **Range:** The value can range from **1 to 65,535**.
- **Example:** If $RefClock = 100\text{ MHz}$ and you set $CD = 3$, the clock frequency after this stage drops to approximately $33.33\text{ MHz}$.

**2. BDIV (Baud Rate Divider)**

- **What it is:** It is the value stored in the **BAUDDIV** register.
- **Function:** This is the **second stage** of division. It further divides the clock coming out of the CD stage to arrive at the final Baud Rate.
- **Range:** For Zynq-7000, this value must be between **4 and 255**.
- **Logic:** The formula uses $(BDIV + 1)$ because the hardware starts counting from zero.

---

```
#include <stdio.h>
#include "xparameters.h"
#include "xuartps.h"
#include "xil_printf.h"

XUartPs Uart_PS;                    /* Instance of UART */

int main() {
   XUartPs_Config *Config;
   s32 Status;

   // 1. UART configuration lookup (UART 0 ke liye)
   Config = XUartPs_LookupConfig(XPAR_XUARTPS_0_DEVICE_ID);

   // 2. Initialize the UART driver
   Status = XUartPs_CfgInitialize(&Uart_PS, Config, Config->BaseAddress);
   if (Status != XST_SUCCESS) return XST_FAILURE;

   // 3. SET BAUD RATE TO 2 MBPS
   // Driver khud CD aur BDIV calculate karke registers mein bhar dega
   Status = XUartPs_SetBaudRate(&Uart_PS, 2000000);
   if (Status != XST_SUCCESS) {
      xil_printf("Baud rate not supported!\n");
      return XST_FAILURE;
   }

   // 4. Test Message
   while(1) {
      xil_printf("Testing 2 Mbps speed on Zynq PS-UART...\r\n");
      for(int i=0; i<1000000; i++); // Small delay
   }

   return 0;
}
```

**1. Header Files**

C

```
#include <stdio.h>

#include "xparameters.h"

#include "xuartps.h"

#include "xil_printf.h"
```

- **stdio.h**: Standard Input/Output library for general C functions.
- **xparameters.h**: This file contains the hardware address map. It defines the Base Address and Device IDs for all peripherals (like UART) enabled in your Vivado design.
- **xuartps.h**: The specific driver for the Zynq **P**rocessor **S**ystem **UART**. It contains all the functions needed to control the hardware.
- **xil_printf.h**: A lightweight version of printf designed for embedded systems to save memory.

**2. Instance Declaration**

C

```
XUartPs Uart_PS;
```

- This declares a **structure** that acts as the handle for your UART. It stores the current state, configuration, and status of the UART hardware.

**3. Main Function & Configuration Lookup**

C

```
int main() {

  XUartPs_Config *Config;

  s32 Status;


  Config = XUartPs_LookupConfig(XPAR_XUARTPS_0_DEVICE_ID);
```

- **XUartPs_Config**: A pointer used to hold the static configuration data of the UART.
- **XUartPs_LookupConfig**: This function searches the hardware database (xparameters.h) to find the entry for UART 0. It retrieves the hardware's base address (e.g., 0xE0000000).

**4. Initialization**

C

```
  Status = XUartPs_CfgInitialize(&Uart_PS, Config, Config->BaseAddress);

  if (Status != XST_SUCCESS) return XST_FAILURE;
```

- **XUartPs_CfgInitialize**: This "wakes up" the hardware. It links your software instance (Uart_PS) to the physical hardware address and resets the UART to a default state.
- **Error Check**: If the hardware is not responding or is missing, the code will stop here.

**5. Setting the 2 Mbps Baud Rate**

C

```
  Status = XUartPs_SetBaudRate(&Uart_PS, 2000000);
```

- **Functionality**: This is where the **CD** and **BDIV** values are calculated.

- Instead of you manually writing to registers, this function reads the input clock frequency, calculates the best integer values for the dividers to reach 2,000,000 bps, and writes them to the **BAUDGEN** and **BAUDDIV** registers automatically.

**6. Validation**

C

```
  if (Status != XST_SUCCESS) {
    xil_printf("Baud rate not supported!\n");
    return XST_FAILURE;
  }
```

- UART communication has an "Error Tolerance." If the 100MHz clock cannot produce 2 Mbps within a safe margin (usually < 5% error), this function returns a failure to prevent corrupted data.

**7. Infinite Loop and Printing**

C

```
  while(1) {
    xil_printf("Testing 2 Mbps speed on Zynq PS-UART...\r\n");
```

- **while(1)**: An infinite loop to keep the program running.
- **xil_printf**: Sends the string to the UART **Transmit FIFO** (Buffer). From there, the hardware shifts it out bit-by-bit at 2 Mbps.
- **\r\n**: Carriage Return and Line Feed to ensure the message starts on a new line on your PC screen.

**8. Software Delay**

C

```
    for(int i=0; i<1000000; i++); // Small delay
  }
  return 0;
}
```

- **Why is this here?**: The CPU is much faster than the UART. At 2 Mbps, the UART can only send a certain amount of characters per second.
- This loop makes the CPU "wait" by counting to 1,000,000, giving the UART enough time to empty its buffer before the next message is sent.

---

Next Step:

Would you like me to show you how to configure Tera Term on your PC to listen at this specific 2,000,000 baud rate? Most standard tools default to 115,200 and will show garbage data otherwise.

# Flash

```
#include "xparameters.h"
#include "xqspips.h"
#include "xil_printf.h"

XQspiPs Qspi;
u8 RB[10];
int main() {
  // Initialization
  XQspiPs_Config *Cfg = XQspiPs_LookupConfig(XPAR_XQSPIPS_0_DEVICE_ID);
  XQspiPs_CfgInitialize(&Qspi, Cfg, Cfg->BaseAddress);
  XQspiPs_SetOptions(&Qspi, XQSPIPS_MANUAL_START_OPTION | XQSPIPS_FORCE_SSELECT_OPTION);

  // 1. Write Enable & Enter 4-Byte Mode
  u8 WREN = 0x06, E4B = 0xB7;
  XQspiPs_PolledTransfer(&Qspi, &WREN, NULL, 1);
  XQspiPs_PolledTransfer(&Qspi, &E4B, NULL, 1);

  // 2. ERASE (Sector 3: 0x00020000)
  u8 Erase[] = {0xDC, 0x00, 0x02, 0x00, 0x00};
  XQspiPs_PolledTransfer(&Qspi, &WREN, NULL, 1);
  XQspiPs_PolledTransfer(&Qspi, Erase, NULL, 5);
  for(volatile int i=0; i<20000000; i++); // Stronger delay for Erase

  // 3. WRITE (0xAB at 0x0002000A)
  u8 Write[] = {0x12, 0x00, 0x02, 0x00, 0x0A, 0xAB};
  XQspiPs_PolledTransfer(&Qspi, &WREN, NULL, 1);
  XQspiPs_PolledTransfer(&Qspi, Write, NULL, 6);
  for(volatile int i=0; i<1000000; i++); // Wait for Write

  // 4. READ (Target Address: 0x0002000A)
  u8 Read[] = {0x13, 0x00, 0x02, 0x00, 0x0A};
  XQspiPs_PolledTransfer(&Qspi, Read, RB, 6);

  xil_printf("Data Read from 10th byte: 0x%X\r\n", RB[5]);

  return 0;
```

}

Bhai, aap bilkul sahi keh rahe ho. Is baar **ek-ek line, ek-ek function, aur ek-ek parameter** ko puri tarah se breakdown karenge. Yeh code Zynq FPGA ke QSPI controller aur ISSI 512Mb Flash memory ke beech ka pura "conversation" hai.

Aaiye shuruat se ek-ek word dekhte hain:

---

**1. Header Files (Software ki Buniyaad)**

- **#include "xparameters.h"**: Isme hardware ke physical address hote hain. Vivado ne jo address QSPI ko diya hai, wo yahan se driver ko pata chalta hai.
- **#include "xqspips.h"**: Yeh Xilinx ka driver hai jo hamein controller se baat karne ke liye functions deta hai.
- **#include "xil_printf.h"**: Yeh processor ko command deta hai ki output UART terminal (computer screen) par dikhaye.

**2. Global Variables (Storage Boxes)**

- **XQspiPs Qspi;**: Yeh ek "Structure" hai. Isme QSPI controller ki current state, base address, aur status save rehta hai. Isse driver ko pata chalta hai ki kis controller ki baat ho rahi hai.
- **u8 RB[10];**: u8 matlab unsigned 8-bit (1 byte). Humne 10 bytes ka ek khali array banaya hai jahan Flash se aane wala data temporary store hoga.

**3. Initialization (Hardware Setup)**

C

```
XQspiPs_Config *Cfg = XQspiPs_LookupConfig(XPAR_XQSPIPS_0_DEVICE_ID);
```

- **XQspiPs_LookupConfig**: Yeh function hardware database mein jaakar QSPI controller ki settings dhoondta hai.
- **XPAR_XQSPIPS_0_DEVICE_ID**: Yeh hardware ki ID hai (0 matlab pehla QSPI controller).

C

```
XQspiPs_CfgInitialize(&Qspi, Cfg, Cfg->BaseAddress);
```

- **&Qspi**: Driver ka pointer.
- **Cfg**: Jo settings humne upar dhoondi.
- **Cfg->BaseAddress**: Yeh controller ka **Main Memory Address** hai. Bina is address ke processor controller ko signal nahi bhej sakta. Yeh line controller ko reset karke use driver se connect kar deti hai.

C

```
XQspiPs_SetOptions(&Qspi, XQSPIPS_MANUAL_START_OPTION | XQSPIPS_FORCE_SSELECT_OPTION);
```

- **MANUAL_START**: Iska matlab data tabhi transfer hoga jab hum PolledTransfer call karenge.
- **FORCE_SSELECT**: SPI mein ek **Chip Select (CS)** pin hoti hai jo Flash ko batati hai ki "Ab tumse baat ho rahi hai". Yeh option CS pin ko poore transfer ke dauran "Low" (Active) rakhta hai taaki message bich mein na toote.

---

**4. Step 1: Write Enable aur 4-Byte Mode (Security aur Rasta)**

C

```
u8 WREN = 0x06, E4B = 0xB7;

XQspiPs_PolledTransfer(&Qspi, &WREN, NULL, 1);

XQspiPs_PolledTransfer(&Qspi, &E4B, NULL, 1);
```

- **WREN = 0x06**: Flash by default locked hoti hai. 0x06 command bhej kar hum uske andar ke "Write Enable Latch" ko set karte hain. Iske bina Erase/Write command chip ignore kar degi.
- **E4B = 0xB7**: Aapki chip **512Mb** ki hai. 3-byte address se sirf 128Mb tak pahunch sakte hain. 0xB7 command chip ko bolta hai: **"Abse 4-byte address accept karo"**. Isi se puri memory ka rasta khulta hai.
- **&Qspi**: Controller handle.
- **&WREN / &E4B**: Jo data bhejna hai uska address.
- **NULL**: Hamein badle mein kuch read nahi karna, isliye katora (buffer) khali hai.
- **1**: Sirf 1 byte ka transfer hai.

---

**5. Step 2: Sector Erase (Purana Saaf Karna)**

C

```
u8 Erase[] = {0xDC, 0x00, 0x02, 0x00, 0x00};

XQspiPs_PolledTransfer(&Qspi, Erase, NULL, 5);
```

- **0xDC**: Yeh 4-byte Sector Erase command hai.
- **0x00, 0x02, 0x00, 0x00**: Yeh **Address** hai (Sector 3). Flash mein likhne se pehle us jagah ko 0xFF (Erase) karna zaroori hai.
- **5**: Humne 5 bytes bheji hain (1 byte Cmd + 4 bytes Addr).
- **for loop**: Erase hone mein kafi waqt lagta hai (0.5 to 1 second). Yeh loop processor ko "Busy" rakhta hai taaki wo agla kaam shuru na kar de jab tak chip saaf ho rahi hai.

---

**6. Step 3: Write Data (Naya Likhna)**

C

```
u8 Write[] = {0x12, 0x00, 0x02, 0x00, 0x0A, 0xAB};

XQspiPs_PolledTransfer(&Qspi, Write, NULL, 6);
```

- **0x12**: Yeh 4-byte Page Program (Write) command hai.
- **0x00, 0x02, 0x00, 0x0A**: Address (Sector 3 ki **10th byte**).
- **0xAB**: Asli **Data** jo hum likh rahe hain.
- **6**: Yahan 6 bytes bheji gayi hain (1 Cmd + 4 Addr + 1 Data).
- **for loop**: Page likhne ke liye chip ko 1-2 millisecond chahiye hote hain.

---

**7. Step 4: Read Data (Check Karna)**

C

```
u8 Read[] = {0x13, 0x00, 0x02, 0x00, 0x0A};

XQspiPs_PolledTransfer(&Qspi, Read, RB, 6);
```

- **0x13**: Yeh 4-byte Read command hai.
- **RB**: Yeh hamara array hai jahan Flash apna data dalegi.
- **6**: Sabse zaroori cheez! Humne 5 bytes bheji, lekin transfer **6** bytes ka kiya.

- **RB[5] Kyun?**:
    1. Pehli 5 bytes bhejte waqt Flash sirf sun rahi hoti hai aur kachra data (0x00/0xFF) wapas bhejti hai.
    2. **6th byte** par Flash apna asli data (0xAB) chip se nikaal kar bhejti hai.
    3. Isliye array mein index 5 (yani 6th position) par hamein data milta hai.

---

**8. Final Result**

- **xil_printf(...)**: Terminal par dikhayega: Data Read from 10th byte: 0xAB.
- **return 0;**: Matlab program bina kisi galti ke khatam hua.

**Bhai ab kuch chhuta?** Maine ek-ek parameter explain kiya hai. Kya ab aapko address nikalna ya dummy cycles ka logic samajhna hai?

I understand. Let's break down this entire QSPI Flash operation code in English, line by line, explaining every single parameter and the hardware logic behind it.

---

**1. Header Files (The Software Foundation)**

- **#include "xparameters.h"**: This file acts as a map of your hardware. It contains the physical addresses defined in Vivado. The driver needs this to know exactly where the QSPI controller is located in the processor's memory map.
- **#include "xqspips.h"**: This is the dedicated Xilinx QSPI driver. It provides the high-level functions (like PolledTransfer) so you don't have to write raw register-level code.
- **#include "xil_printf.h"**: A lightweight version of printf used to send text to the UART terminal so you can see the results on your computer screen.

---

**2. Global Variables (The Storage Boxes)**

- **XQspiPs Qspi;**: This is an "Instance Structure." It stores the current state, configuration, and status of the QSPI controller. Think of it as a dedicated "handle" for this specific piece of hardware.
- **u8 RB[10];**: u8 stands for Unsigned 8-bit Integer (1 byte). This is a 10-byte array (buffer) used to temporarily store the data that comes back from the Flash chip during a Read operation.

---

**3. Initialization (Hardware Setup)**

C

```
XQspiPs_Config *Cfg = XQspiPs_LookupConfig(XPAR_XQSPIPS_0_DEVICE_ID);
```

- **LookupConfig**: This function searches the hardware database for the QSPI controller settings using its unique ID (0 for the first controller).

C

```
XQspiPs_CfgInitialize(&Qspi, Cfg, Cfg->BaseAddress);
```

- **&Qspi**: The pointer to our controller instance.
- **Cfg->BaseAddress**: This is the **Main Memory Address** where the controller lives. This function resets the hardware and connects the software driver to that specific physical location.

C

```
XQspiPs_SetOptions(&Qspi, XQSPIPS_MANUAL_START_OPTION | XQSPIPS_FORCE_SSELECT_OPTION);
```

- **MANUAL_START**: Ensures data is only transferred when we explicitly call the transfer function.
- **FORCE_SSELECT**: In SPI, there is a **Chip Select (CS)** pin. This option keeps that pin "Low" (Active) throughout the entire multi-byte transfer. If the CS pin toggles in the middle of a command, the Flash chip will reset its internal logic and fail the operation.

---

**4. Step 1: Write Enable & 4-Byte Mode (Security & Access)**

C

```
u8 WREN = 0x06, E4B = 0xB7;

XQspiPs_PolledTransfer(&Qspi, &WREN, NULL, 1);

XQspiPs_PolledTransfer(&Qspi, &E4B, NULL, 1);
```

- **WREN = 0x06**: Flash memory is write-protected by default. This command sets the "Write Enable Latch" (WEL) bit. Without sending this 0x06 first, the chip will ignore any Erase or Write commands.
- **E4B = 0xB7**: Your chip is **512Mb**. Standard 3-byte addressing can only reach up to 128Mb. The 0xB7 command tells the chip: **"Switch to 4-byte addressing mode."** This opens the path to access the entire memory range beyond the 128Mb limit.
- **NULL**: Since we are only sending a command and not expecting data back, the "Receive Buffer" is set to NULL.
- **1**: We are sending exactly 1 byte.

---

**5. Step 2: Sector Erase (Cleaning the Memory)**

C

```
u8 Erase[] = {0xDC, 0x00, 0x02, 0x00, 0x00};

XQspiPs_PolledTransfer(&Qspi, Erase, NULL, 5);
```

- **0xDC**: This is the specific Opcode for a **4-byte Sector Erase**.
- **0x00, 0x02, 0x00, 0x00**: This is the **Address** (Sector 3). Flash memory cannot overwrite a '0' with a '1'; you must erase the whole sector (resetting it to 0xFF) before writing new data.
- **5**: The total count: 1 byte for the Command + 4 bytes for the Address.
- **for loop**: Erasing is a physical process that takes time (up to 1 second). The processor is much faster, so we use a delay loop to wait until the chip has finished erasing.

---

**6. Step 3: Write Data (Page Program)**

C

```
u8 Write[] = {0x12, 0x00, 0x02, 0x00, 0x0A, 0xAB};

XQspiPs_PolledTransfer(&Qspi, Write, NULL, 6);
```

- **0x12**: The Opcode for **4-byte Page Program**.
- **0x00, 0x02, 0x00, 0x0A**: The specific address (Sector 3, **10th byte**).
- **0xAB**: The actual **Data** value we want to store.
- **6**: The byte count is crucial here. We send 1 byte for Command, 4 for Address, and 1 for Data. If you put 5 here, the 0xAB value would never leave the FPGA.

---

**7. Step 4: Read Data (Verification)**

C

```
u8 Read[] = {0x13, 0x00, 0x02, 0x00, 0x0A};

XQspiPs_PolledTransfer(&Qspi, Read, RB, 6);
```

- **0x13**: The Opcode for **4-byte Read**.
- **RB**: Our Read Buffer array.
- **6**: Why 6? Because SPI is full-duplex.
    1. During the first 5 bytes (Command + Address), the Flash chip is receiving instructions and sending back "dummy" or "garbage" data (usually 0x00 or 0xFF).
    2. Only on the **6th byte** does the Flash chip start outputting the actual stored data (0xAB).

- **RB[5]**: Since C arrays start at index 0, the 6th byte received is stored at RB[5].

---

**8. The Final Result**

- **xil_printf**: Prints the result: Data Read from 10th byte: 0xAB.
- **return 0;**: Indicates successful program completion.

Every byte, every parameter, and every wait loop serves a specific purpose in syncing the fast FPGA processor with the slower Flash memory chip. Does this clear up every part of the code for you? Would you like to dive deeper into how **dummy cycles** work for faster read commands?